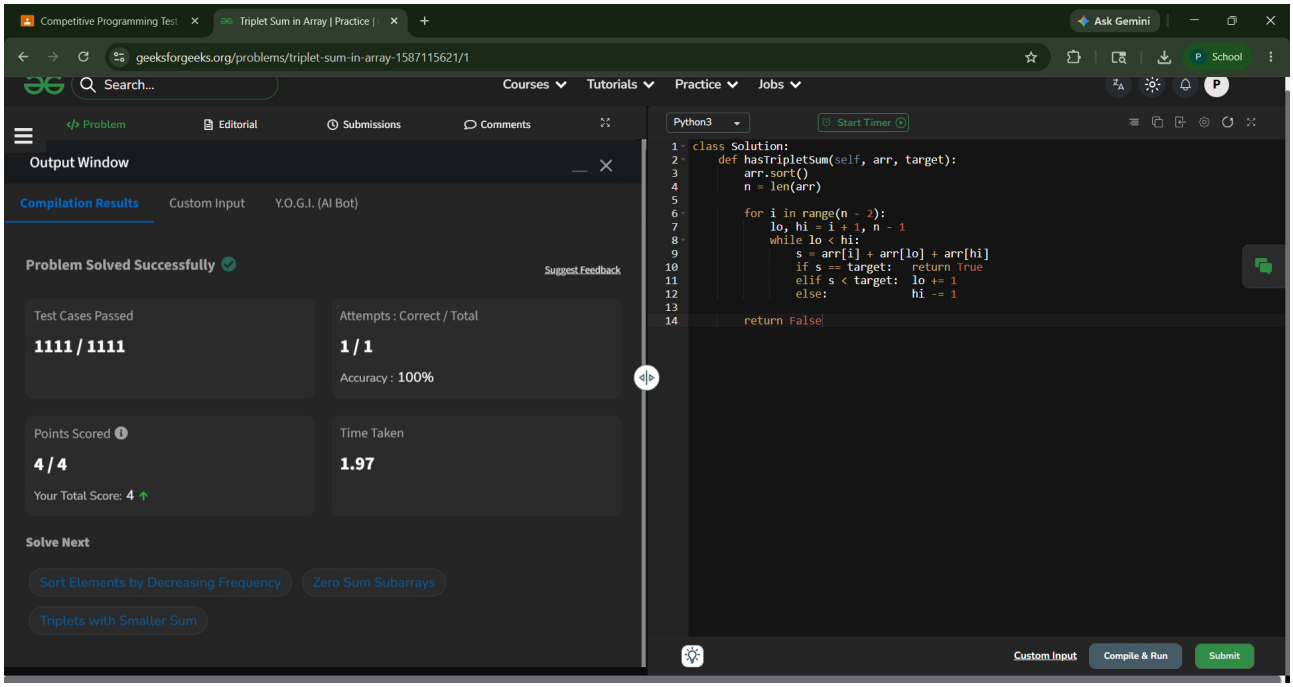
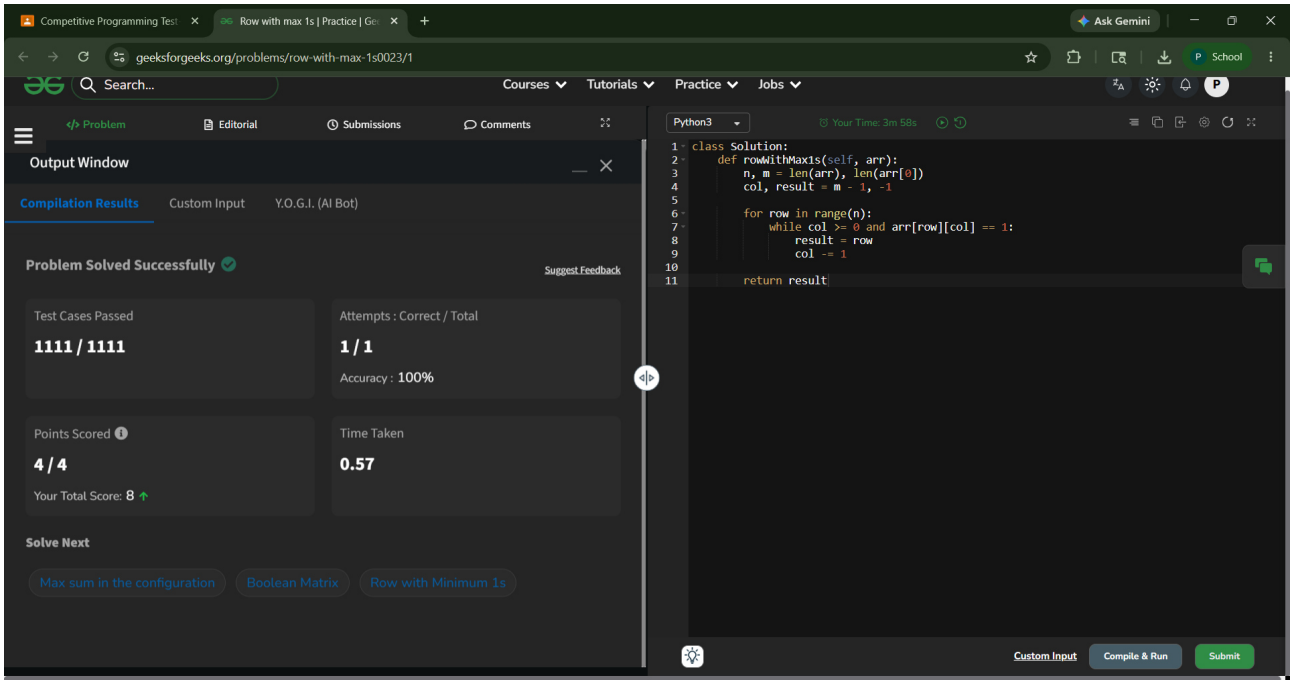



**Logic Used:** Sort the array, then fix one element at a time using a loop. For each fixed element `arr[i]`, use two pointers (`lo` and `hi`) starting from `i+1` and `n-1` to find if any pair sums to `target - arr[i]`. If the current sum is too small, move `lo` right; if too large, move `hi` left.
**Time Complexity:**

- Sorting → $O(n \log n)$
- Outer loop × two-pointer scan → $O(n^2)$
- **Overall → $O(n^2)$**




**Logic Used:** Start from the **top-right corner** `(0, m-1)`. For each row, move left as long as the current cell is `1`, recording the row index. Since rows are sorted, once the column pointer moves left it never needs to go right again — each row either extends the leftmost `1` boundary or doesn't.
**Time Complexity:**

- Row pointer moves down `n` steps, column pointer moves left at most `m` steps total.
- **Overall → `O(n + m)`**